

21 ADMIN

A monorepo SaaS for payroll, employee loans, and recurring community funds — with a separate, dedicated portal for treasurers and fund members who are never company users at all.

Next.js 14 (App Router) Fastify PostgreSQL Prisma Redis + BullMQ pnpm + Turborepo

01 Overview 02 Tenancy & Auth 03 Roles & Access 04 Domain Modules 05 Billing & Entitlements 06 Deployment 07 What's Next

01 OVERVIEW

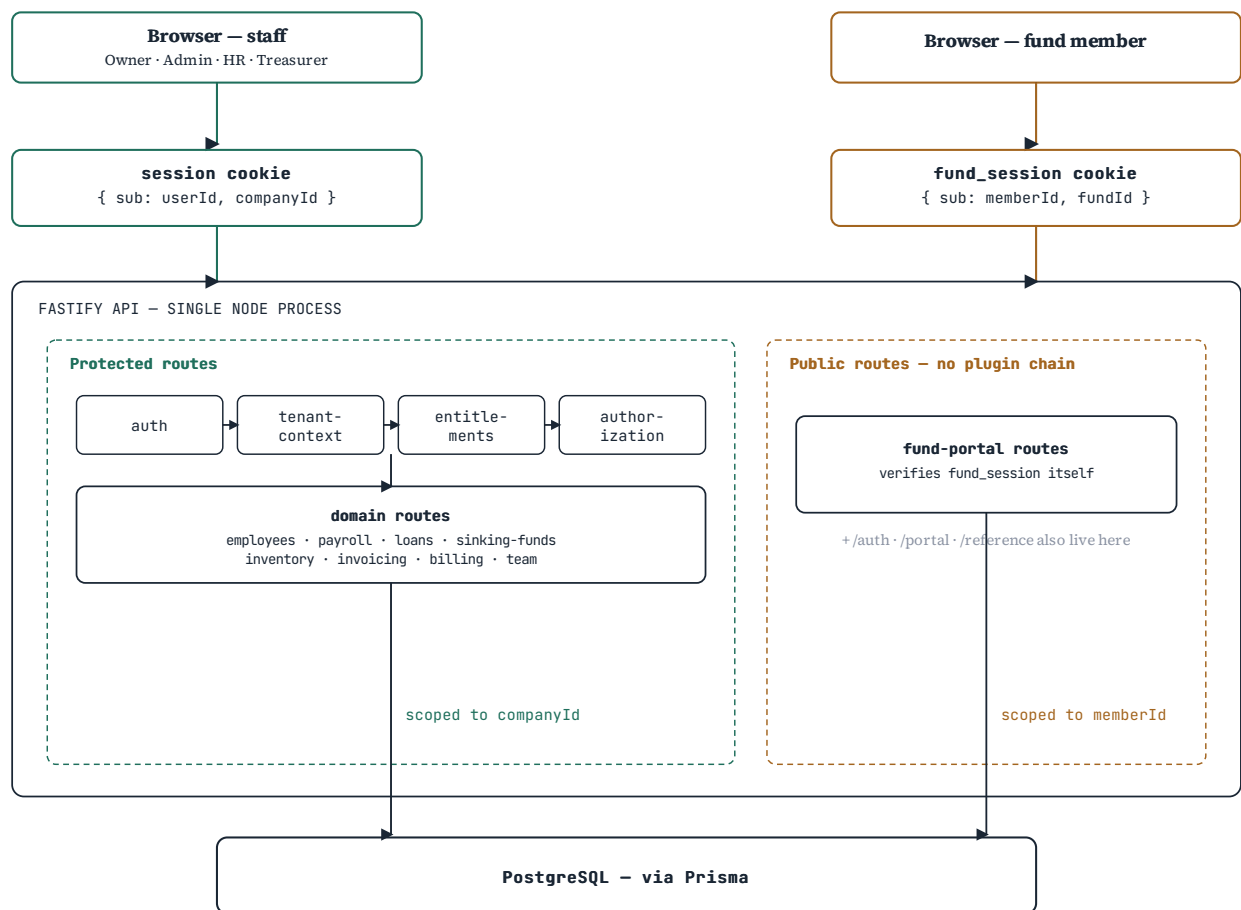
The codebase is a pnpm workspace with two runtime apps and two shared packages, built and run through Turborepo. `apps/web` is a single Next.js app serving three distinct audiences from one deployment: the marketing site, the owner/staff dashboard, and the employee & fund-member portals. `apps/api` is one Fastify process exposing every domain module behind a single REST surface.

Every table in the system hangs off `Company` — the tenant boundary. An owner can hold a `CompanyMembership` in more than one company and switch the active one from the sidebar; the session cookie's `companyId` claim is what actually scopes every query on the API side, not anything client-supplied.

LAYER	PACKAGE	RESPONSIBILITY
Web	<code>apps/web</code>	Marketing, dashboard, employee & fund portals — one Next.js deployment
API	<code>apps/api</code>	Fastify REST API — auth, RBAC, every domain module, file uploads
Data	<code>packages/db</code>	Prisma schema + generated client, single Postgres database
Types	<code>packages/shared-types</code>	Types & constants shared by both apps — module keys, plans, enums

02 TENANCY & AUTH

There is no single "logged in" concept — there are two independent session systems that happen to share a server process. A staff member's `session` cookie is a JWT over `{ sub: userId, companyId }` and is checked by a four-stage plugin chain on every protected route. A fund member's `fund_session` cookie is a JWT over `{ sub: memberId, fundId }` and is verified by hand inside the fund-portal route handlers, which sit in the public route group entirely outside that chain — a fund member is a `SinkingFundMember` row, never a `User`, so the RBAC pipeline built for staff doesn't apply to them and isn't asked to.



The staff session and the fund-member session are verified by completely different code paths inside one Fastify process. A fund member can never accidentally pass the staff RBAC gate, because it never runs for that request at all.

Why two systems

A sinking fund is often run by someone with no employees, no payroll, and no interest in creating a staff account for every contributing member — a barangay treasurer, an HOA officer. Reusing the `User` /RBAC model for fund members would mean every member needs a role, a module entitlement check, and a seat on a plan built for a different buyer. Instead a `SinkingFundMember` is its own row with its own `memberCode`, password hash, and login page at `/fund-slug/login` — indistinguishable in the URL bar from a company's own login, resolved by whichever session the server can verify first.

03 ROLES & ACCESS

Six roles, enforced server-side on every route via `app.requireRole(...)` — never trusted from the client. `MEMBER` is the odd one out: it's what an `Employee` looks like as a `User`, and it has no dashboard access at all, only the employee portal.

ROLE	TEAM & SETTINGS	EMPLOYEES	PAYROLL	LOANS & FUNDS	INVOICING	INVENTORY	BILLING
OWNER	✓	✓	✓	✓	✓	✓	✓
ADMIN	✓	✓	—	✓	—	✓	—
HR_MANAGER	—	✓	✓	—	—	—	—
HR_ASSISTANT *	—	✓	—	—	—	—	—
TREASURER	—	—	—	✓	✓	—	—

ROLE	TEAM & SETTINGS	EMPLOYEES	PAYROLL	LOANS & FUNDS	INVOICING	INVENTORY	BILLING
MEMBER †	—	—	—	—	—	—	—

* HR_ASSISTANT can edit an employee record but not their salary, salary currency, or bank details — those fields are blocked server-side regardless of what the request body contains.

† MEMBER has no dashboard route at all; hitting one redirects straight to `/slug/portal`.

04 DOMAIN MODULES

Modules are switched on per company via an `Entitlement` row (`{ companyId, module, active }`), and gated on the API with `app.requireModule()` in addition to the role check above.

MODULE	KEY	STATUS	WHAT IT DOES
Employees	employees	Live	Directory, bank details, attendance, shift schedules, self-service portal
Payroll	payroll	Live	Runs by pay period, auto-calculated deductions, payslips in the portal
Loans	loans	Live	Funded by company cash or a sinking fund; interest, term, repayment tracking
Sinking Funds	sinking-funds	Live	Per-member dues tracking, its own member portal, QR payments — sold as a standalone add-on
Inventory	inventory	Coming soon	Stock levels by SKU, low-stock alerts
Invoicing	invoicing	Coming soon	Customer invoices, paid / sent / overdue status

05 BILLING & ENTITLEMENTS

A company's `plan` field picks which core modules are active; Sinking Funds sits outside that entirely as its own toggle, because it's just as often the *only* thing a customer wants.

PLAN	PRICE	CORE MODULES
Starter	Free	Employees
Growth	\$29/mo	Employees, Payroll, Loans
Business	\$59/mo	Employees, Payroll, Loans, Inventory, Invoicing
Sinking Funds (add-on)	metered	Priced per active fund member, independent of plan

Switching plans re-writes the company's core `Entitlement` rows in one transaction; the Sinking Funds row is untouched by that write. Payment collection isn't wired yet — plan changes apply instantly, no charge is made.

06 DEPLOYMENT

Scaffolded for Google Cloud Run, not yet applied. Turborepo drives both build and dev locally; there's no CI pipeline wired to the Cloud Build config yet.

COMPONENT	TARGET	FILE
Web	Cloud Run service	<code>infra/cloud-run-web.yaml</code>
API	Cloud Run service	<code>infra/cloud-run-api.yaml</code>
Build	Cloud Build	<code>infra/cloudbuild.yaml</code>
Infra	Terraform (stub)	<code>infra/terraform/</code>

`build` Inventory and Invoicing — currently placeholder pages behind their entitlement flags

`wire` Real payment collection — Xendit GCash checkout + payout, env vars already scaffolded but unset

`wire` The three BullMQ queues in `apps/api/src/queues/` — invoice PDF, payroll run, report export are all empty workers today

`add` Automated overdue reminders for loan repayments and fund contributions — the due-date data already exists, nothing notifies anyone yet

21admin/business-platform · monorepo

Draft — reflects the codebase as of this writing, not a fixed spec